CHAPTER 4

Kick-Starting Robot Programming Using ROS

The last three chapters discussed the prerequisites for programming a robot using the Robot Operating System (ROS). We discussed the basics of Ubuntu Linux, bash commands, the basic concepts of C++ programming, and the basics of Python programming. In this chapter, we start working with ROS. Before discussing ROS concepts, let's discuss robot programming and how we do it. After this, we learn more about ROS, how to install ROS, and its architecture.

After this, we look at ROS concepts, ROS command tools, and ROS examples to demonstrate ROS capabilities. After that, we discuss the basics of ROS GUI tools and the Gazebo simulator. In the end, we learn how to set up a TurtleBot 3 simulator in ROS.

What Is Robot Programming?

As you know, a robot is a machine with sensors, actuators (motors), and a computing unit that behaves based on user controls, or it can make its own decisions based on sensors inputs. We can say the brain of the robot is a computing unit. It can be a microcontroller or a PC. The decision making

© Lentin Joseph 2018
L. Joseph, *Robot Operating System for Absolute Beginners*,
https://doi.org/10.1007/978-1-4842-3405-1_4

Professor & Head
Dept of Robotics & Artificial Intelligence
Bangalore Institute of Technology
K.R. Road, V.V. Puram,
Bangalore-560004

www.electronicbo.com

and actions of the robot completely depends on the program running the robot's brain. This program can be firmware running on a microcontroller, or C/C++ or Python code running on a PC or a single board computer, like the Raspberry Pi. Robot programming is the process of making the robot work from writing a program for the robot's brain (i.e., the processing unit).

Figure 4-1 shows a general block diagram of a robot, including the part where it programs.

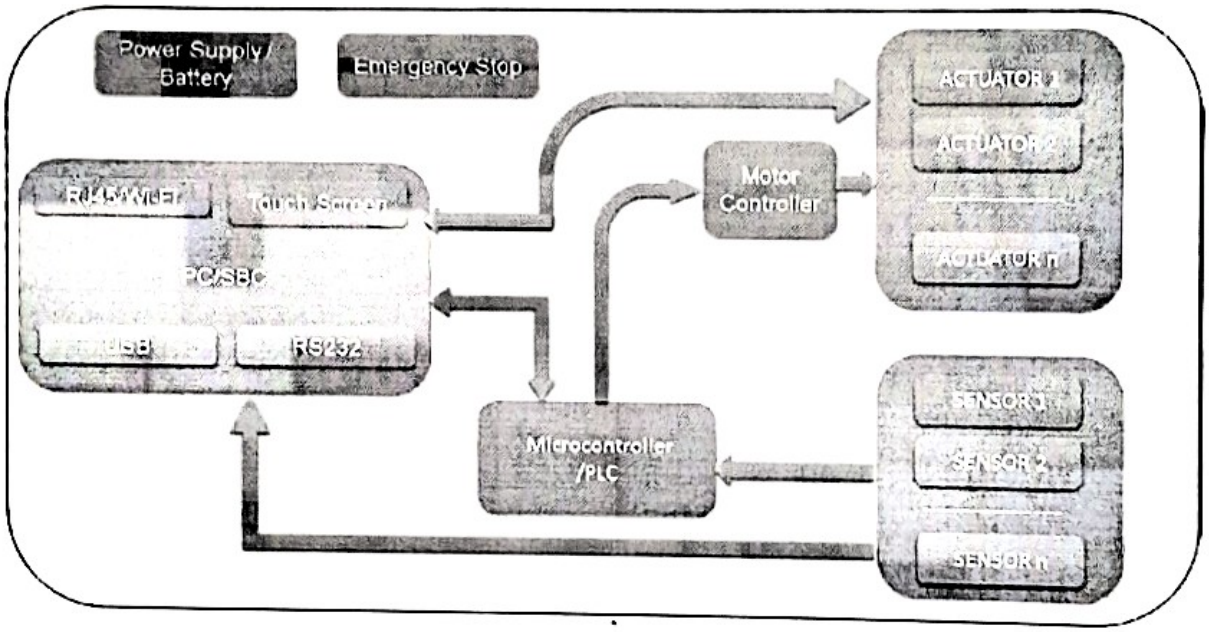


Figure 4-1. *General block diagram of a robot*

The main components of any robot are the actuators and the sensors. Actuators move a robot's joints, providing rotary or linear motion. Servo, Stepper, and DC gearmotors are actuator brands. Sensors provide the robot's state and environment. Examples of robot sensors include wheel encoders, ultrasonic sensors, and cameras.

Actuators are controlled by motor controllers and interface with a microcontroller/PLC (programmable logic controller). Some actuators are directly controlled through a PC's USB. Sensors also interface with a microcontroller or PC. Ultrasonic sensors and infrared sensor interface with a microcontroller. High-end sensors like cameras and laser scanners

can interface directly with the PC. There is a power supply/battery to power all the robotic components. There is an emergency stop push-button to stop/reset the robot's operation. The two major parts in which to program inside a robot is a PC and a microcontroller/PLC. PLCs are mainly using in industrial robots.

In short, we can say robot programming is programming the PC/SBC and microcontroller/PCL inside robot for performing a specific application using actuators and feedback from various sensors. The robot applications includes pick and place of object, moving robot from A to B. A variety of programming languages can program robots. C/C++, Python, Java, C #, and so forth are used with PCs. Microcontrollers use Embedded C, the Wiring language (based on C++), which is used in Arduino, and Mbed programming (`https://os.mbed.com`). Industrial robot applications use SCADA or vendors' proprietary programming languages, such as ABB and KUKA. This programming is done from the industrial robot's teach pendant. RAPID is the programming language used in ABB industrial robots to automate robotic applications.

Robotic programming creates intelligence in the robot for self-decision making, implementing controllers like PID to move joints, automating repeated tasks, and creating robotic vision applications.

Why Robot Programming Is Different

Robot programming is a subset of computer programming. Most robots have a "brain" that can make decisions. It can be a microcontroller or a PC. The differences between robot programming and conventional programming are the input and output devices. The input devices include robot sensors, teach pendants, and touch screens, and the output devices include LCD displays and actuators.

Any of the programming languages can program robots, but good community support, performance, and prototyping time make C++ and Python the most commonly used.

www.electronicbo.com

The following are some of the features needed for programming a robot.

- *Threading*: As seen in the robot block diagram, there are a number of sensors and actuators in a robot. We may need a multithreaded compatible programming language in order to work with different sensors and actuators in different threads. This is called *multitasking*. Each thread can communicate with each other to exchange data.

- *High-level object-oriented programming*: As you already know, object-oriented programming languages are more modular and code can easily reused. The code maintenance is also easy compared to non-object-oriented programming languages. These qualities create better software for robots.

- *Low-level device control*: The high-level programming langauges can also access low-level devices such as GPIO (general purpose input/output) pins, Serial ports, parallel ports, USB, SPI, and I2C. Programming languages like C/C++ and Python can work with low-level devices, which is why these languages prefer single-board computers like the Raspberry Pi and Odroid.

- *Ease of prototyping*: The easiness in prototyping a robot algorithm is definitely a choice in the selection of programming language. Python is a good choice in prototyping robot algorithms quickly.

- *Interprocess communication*: A robot has lot of sensors and actuators. We can use multithreading architecture or write an independent program for doing each task; for example, one program takes images from a camera and detects a face, another program sends

data to an embedded board. These two programs can communicate with each other to exchange data. This feature creates multiple programs instead of a multithreading system. The multithreading system is more complicated than running multiple programs in parallel. Socket programming is an example of interprocess communication.

- *Performance*: If we work with high-bandwidth sensors, such as depth cameras and laser scanners, the computing resources needed to process the data is obviously high. A good programming language can only allocate appropriate computing resource without loading the computing resource. The C++ langauges is a good choice to handle these kind of scenerio.

- *Community support*: When choosing any programming language for robot programming, make sure that there is enough community support for that language, including forums and blogs.

- *Availability of third-party libraries*: The availability of third-party libraries can make our development easy; for example, if we want to do image processing, we can use libraries like OpenCV. If your programming language has OpenCV support, it is easier to do image processing applications.

- *Existing robotics software framework support*: There are existing robotics software frameworks such as ROS to program robots. If your programming language has ROS support, it is easier to prototype a robot application.

www.electronicbo.com

Getting Started with ROS

So far, we have discussed robot programming and how it is different from other computer programming. In this section, we look at a unique software platform for programming robots: the Robot Operating System, or ROS (`www.ros.org`).

ROS is a free and open source robotics software framework that is used in both commercial and research applications. The ROS framework provides the following robot-programming capabilities.

- *Message passing interface between processes.* ROS provides a message passing interface to communicate between two programs or processes. For example, a camera processes an image and finds coordinates in the image, and then these coordinates are sent to a tracker process. The tracker process does the tracking of the image by using motors. As mentioned, this is one of the features needed to program a robot. It is called *interprocess communication* because two processes are communicating with each other.
- *Operating system–like features.* As the name says, ROS is not a real operating system. It is a meta operating system that provides some operating system functionalities. These functionalities include multithreading, low-level device control, package management, and hardware abstraction. The hardware abstraction layer enables programmers to program a device. The advantage is that we can write code for a sensor that works the same way with different vendors. So, we don't need to rewrite the code when we use a new sensor. Package management helps users organize software in units called *packages*. Each package has

source code, configuration files, or data files for a specific task. These packages can be distributed and installed on other computers.

- *High-level programming language support and tools.* The advantage of ROS is that it supports popular programming languages used in robot programming, including C++, Python, and Lisp. There is experimental support for languages such as C #, Java, Node.js, and so forth. The complete list is at `http://wiki.ros.org/Client%20Libraries`. ROS provides client libraries for these languages, meaning the programmer can get ROS functionalities in the languages mentioned. For example, if a user wants to implement an Android application that is using ROS functionality, the rosjava client library can be used. ROS also provides tools to build robotics applications. With these tools, we can build many packages with a single command. This flexibility helps programmers spend less time in creating build systems for their applications.

- *Availability of third-party libraries.* The ROS framework is integrated with most popular third-party libraries; for example, OpenCV (`https://opencv.org`) is integrated for robotic vision, and PCL (`http://pointclouds.org`) is integrated for 3D robot perception. These libraries make ROS stronger, and the programmer can build powerful applications on top of it.

- *Off-the-shelf algorithms.* This is a useful feature. ROS has implemented popular robotics algorithms such as PID (`http://wiki.ros.org/pid`); SLAM (Simultaneous Localization and Mapping) (`http://wiki.ros.org/gmapping`); and path planners such as A*, Dijkstra

www.electronicbo.com

(http://wiki.ros.org/global_planner), and AMCL (Adaptive Monte Carlo Localization) (http://wiki.ros.org/amcl). The list of algorithm implementations in ROS continuous. The off-the shelf algorithms reduce development time for prototyping a robot.

- *Ease in prototyping*. One advantage of ROS is off-the-shelf algorithms. Along with that, ROS has packages that can be easily reused with any robot; for example, we can easily prototype our own mobile robot by customizing an existing mobile robot package available in the ROS repository. We can easily reuse the ROS repository because most of the packages are open source and reusable for commercial and research purposes. So, this can reduce robot software development time.

- *Ecosystem/community support*. The main reason for the popularity and development of ROS is community support. ROS developers are all over the world. They actively develop and maintain ROS packages. The big community support includes developers asking questions related to ROS. ROS Answers is a platform for ROS-related queries (https://answers.ros.org/questions/). ROS Discourse is an online forum in which ROS users discuss various topics and publish news related to ROS (https://discourse.ros.org).

- *Extensive tools and simulators*. ROS is built with many command-line and GUI tools to debug, visualize, and simulate robotics applications. These tools are very useful for working with a robot. For example, the Rviz (http://wiki.ros.org/rviz) tool is used for

visualization with cameras, laser scanners, inertial measurement units, and so forth. For working with robot simulations, there are simulators such as Gazebo (`http://gazebosim.org`).

The ROS Equation

The ROS project can be defined in a single equation, as shown in Figure 4-2.



Figure 4-2. The ROS equation

The plumbing is the same as the message passing interface. ROS has many other capabilities, which we explore in upcoming sections.

Robot Programming Before and After ROS

Let's look at the changes to the robotics programming community since the ROS project began.

The History of ROS

The following are some ROS project historic milestones.

- The ROS project started at Stanford University in 2007, led by roboticist Morgan Quigly (`http://people.osrfoundation.org/morgan/`). In the beginning it was a group of software developed for robots at Stanford.

www.electronicbo.com

- Later in 2007, a robotics research startup called Willow Garage (`http://www.willowgarage.com/`) took over the project and coined the name ROS, which stands for Robot Operating System.
- In 2009, ROS 0.4 was released, and a working ROS robot called PR2 was developed.
- In 2010, ROS 1.0 was released. Many of its features are still in use.
- In 2010, ROS C Turtle was released.
- In 2011, ROS Diamondback was released.
- In 2011, ROS Electric Emys was released.
- In 2012, ROS Fuerte was released.
- In 2012, ROS Groovy Galapagos was released.
- In 2012, the Open Source Robotics Foundation (OSRF) takes over the ROS project.
- In 2013, ROS Hydro Medusa was released.
- In 2014, ROS Indigo Igloo was released, this was the first long-term support (LTS) release, meaning updates and support is provided for a long period of time (typically five years).
- In 2015, ROS Jade Turtle was released.
- In 2016, ROS Kinetic Kame was released. It is the second LTS version of ROS.
- In 2017, ROS Lunar Loggerhead was released.
- In May 2018, the twelfth version of ROS, Melodic Morenia, was released.

The timeline of the ROS project and a more detailed history is available in at `www.ros.org/history/`.

Each version of ROS is called a ROS distribution. You may be aware of the Linux distribution, such as Ubuntu, Debian, Fedora, and so forth.

Figure 4-3 shows the complete list of ROS distribution releases (`http://wiki.ros.org/Distributions`).

Distro	Release date	Poster	*Tuturtle*, turtle in tutorial	EOL date
ROS Melodic Morenia	May, 2018	TBD	TBD	May, 2023
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame **(Recommended)**	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)

Figure 4-3. *ROS distributions*

If you are looking for the latest ROS features, you can choose new distributions, and if you are looking for stable packages, you can choose LTS. In Figure 4-3, the recommended distribution is ROS Kinetic Kame. In this book, the examples use Kinetic Kame.

ROS is now developed and maintained by the Open Robotics, previously known as the Open Source Robotics Foundation (`www.osrfoundation.org`).

www.electronicbo.com

Before and After ROS

There was active development in robotics before the ROS project, but there was no common platform and community for developing robotics applications. Each developer created software for their own robot, which in most cases, couldn't be reused for any other robot. Developers had to rewrite code from scratch for each robot, which takes a lot of time. Also, most of the code was not actively maintained, so there was no support for the software. Also, developers needed to implement standard algorithms on their own, which took more time to prototype the robot.

After the ROS project, things changed. Now there is a common platform for developing robotics applications. It is free and open source for commercial and research purposes. Off-the-shelf algorithms are readily available, so there is no longer a need to code. There is big community support, which makes development easier. In short, the ROS project changed the face of robotics programming.

Why Use ROS?

This is common question that developers ask when looking for a platform to program ROS. Although ROS has many features, there are still areas in which ROS can't be used or is not recommended to use. In the case of a self-driving car, for example, we can use ROS to make a prototype, but developers do not recommend ROS to make the actual product. This is due to various issues, such as security, real-time processing, and so forth. ROS may not be a good fit in some areas, but in other areas, ROS is an absolute fit. In corporate robotics research centers and at universities, ROS is an ideal choice for prototyping. And ROS is used in some robotics products after lot of fine-tuning (but not self-driving cars).

A project called ROS 2.0 is developing a much better version of the existing ROS in terms of security and real-time processing (`https://github.com/ros2/ros2/wiki`). ROS 2.0 may become a good choice for robotics products in the future.

Robots and Sensors Supporting ROS

Figure 4-8 shows some of the popular robots that use ROS. A complete list of robots working in ROS is at `http://robots.ros.org`.

The following are the robots shown in Figure 4-8.

a) Pepper (`www.ald.softbankrobotics.com/en/robots/pepper`): A service robot used for assisting people in a variety of ways.

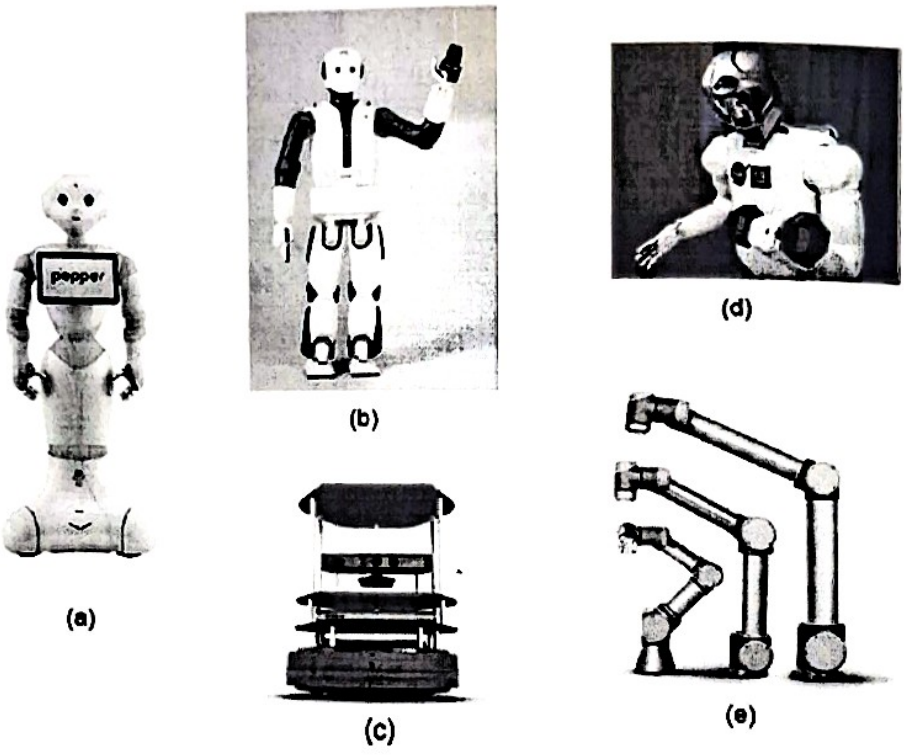


Figure 4-8. *Robots that work in ROS*

b) REEM-C (`http://pal-robotics.com/en/products/reem-c/`): A full-size humanoid robot that is mainly used for research purposes.

c) TurtleBot 2 (`www.turtlebot.com/turtlebot2/`): A simple mobile robot platform that is mainly used for research and educational purposes.

d) Robonaut 2 (`https://robonaut.jsc.nasa.gov/R2/`): A NASA robot designed to automate various tasks on the International Space Station.

e) Universal Robot arm: (`www.universal-robots.com/products/ur5-robot`): One of the popular semi-industrial robots widely used for automating various tasks in manufacturing.

There are also sensors supported by ROS. A complete list of these sensors is available at `http://wiki.ros.org/Sensors` (see Figure 4-9).

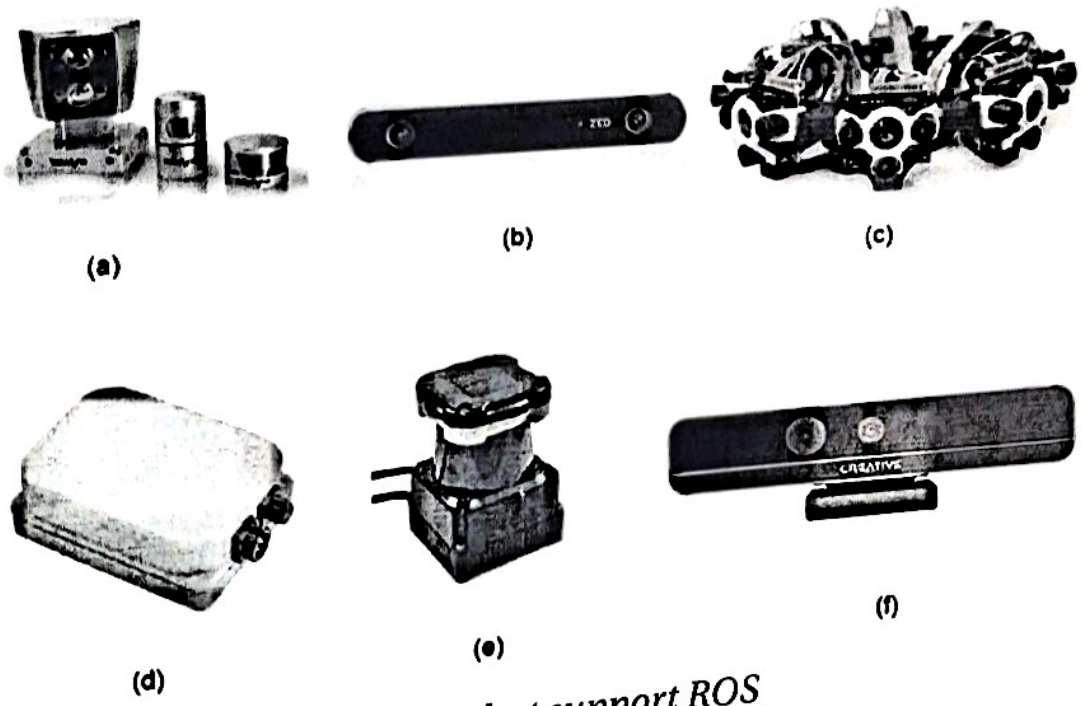


Figure 4-9. *Popular sensors that support ROS*

The following describes each sensor shown in Figure 4-9.

a) Velodyne (`http://velodynelidar.com`): Popular LIDARs mainly used in self-driving cars.

b) ZED Camera (`www.stereolabs.com`): A popular stereo depth camera.

www.electronicbo.com

c) TeraRanger (`www.terabee.com`): A new sensor for depth sensing in 2D and 3D.

d) Xsense MTi IMU (`www.xsens.com/products/`): An accurate IMU solution.

e) Hokuyo Laser (`www.hokuyo-aut.jp/`): A popular laser scanner.

f) Intel RealSense (`https://realsense.intel.com`: A 3D depth sensor for robot navigation and mapping.

Popular ROS Computing Platforms

Figure 4-10 shows a few commonly used ROS-compatible computing platforms.

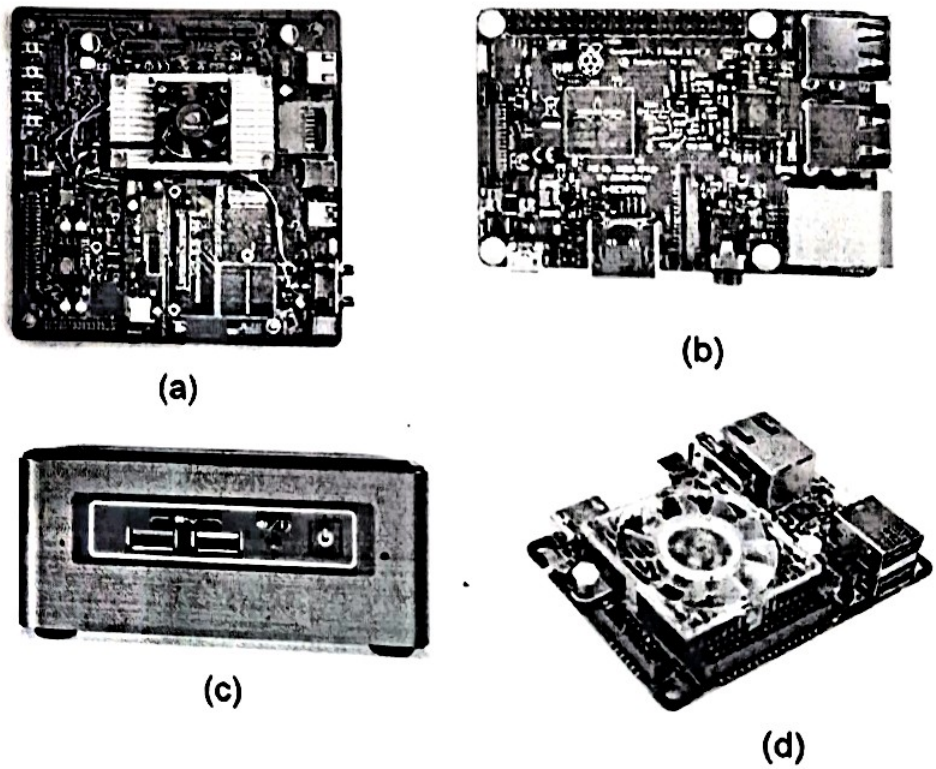


Figure 4-10. *Popular computing units that run ROS*

a) NVDIA TX1/TX2 (www.nvidia.com/en-us/autonomous-machines/embedded-systems-dev-kits-modules/): Capable of running deep learning applications and computational intensive applications. The board has an ARM-based 64-bit processor that can run Ubuntu. This platform is very popular in autonomous robotics applications, especially drones.

b) Raspberry Pi 3 (www.raspberrypi.org/products/raspberry-pi-3-model-b/): Very popular single-board computers for education and research. Robotics is a key area.

c) Intel NUC (www.intel.com/content/www/us/en/products/boards-kits/nuc.html): Based on a x86_64 platform, which is basically a miniature version of a desktop computer.

d) Odroid XU4 (www.hardkernel.com/main/main.php): The Odroid series boards are similar to Raspberry Pi, but it has better configuration and performance. It is based on the ARM architecture.

www.electronicbo.com

ROS Architecture and Concepts

We have discussed ROS, its features, and how to install it. In this section we go deep into ROS architecture and its important concepts. Basically, ROS is a framework to communicate between two programs or process. For example if program A wants to send a data to program B, and B wants to send data to program A, we can easily implement it using ROS. So the question is whether we implement it using socket programming directly. Yes, we can, but if we build more and more programs, it gets complex, so ROS is a good choice for interprocess communication.

Do we really need interprocess communication in a robot? Can we program a robot without it? The answer to the first question is explained in Figure 4-11.

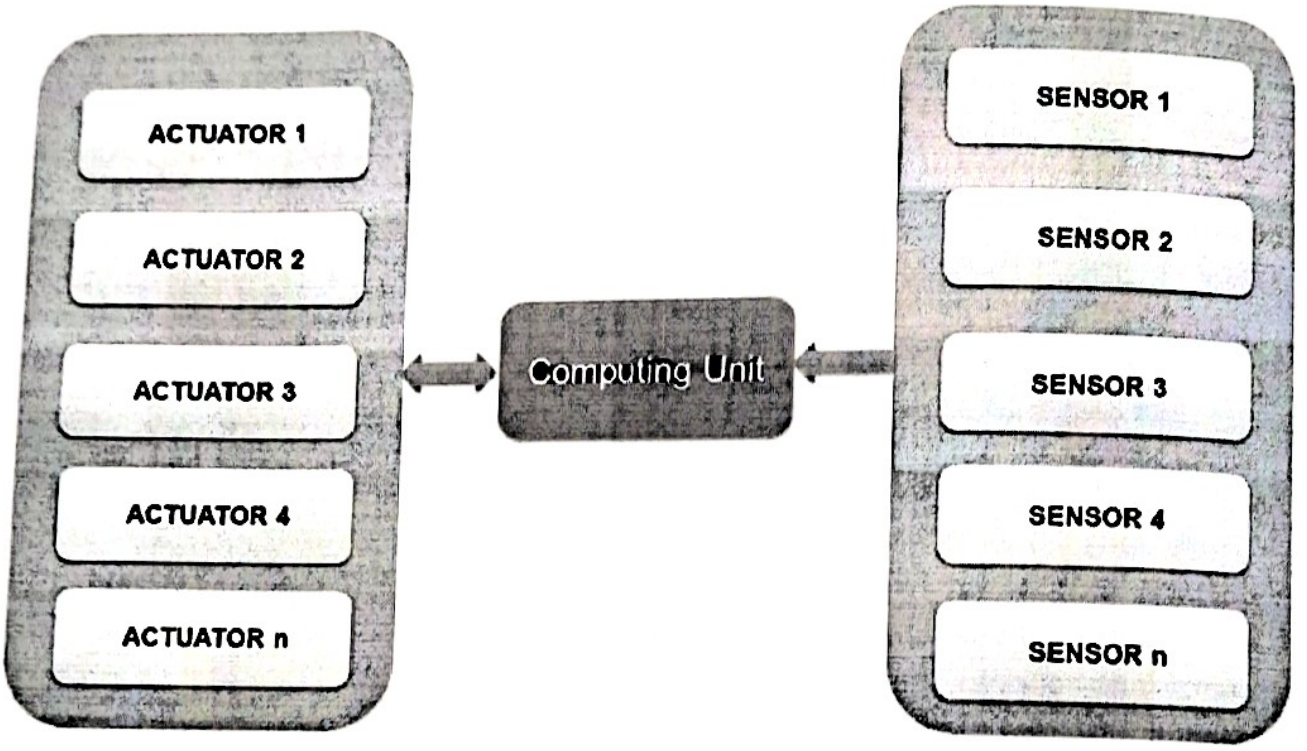


Figure 4-11. *A typical robot block with actuators and sensors*

A robot may have many sensors and actuators, as well as a computing unit. How we can control many actuators and process so much sensor data? Can we do it in a single program? Yes, but that is not a good way of doing it. The better way is, we can write independent programs to handle sensor data and controlling actuators, and often we may need to exchange data between these programs. This is the situation where we use ROS.

So can we program a robot without ROS? Yes, but the complexity of software increases according to the number of actuators and sensors.

Let's see how the communication is happening between two programs in ROS. Figure 4-12 illustrates a basic block diagram of ROS.

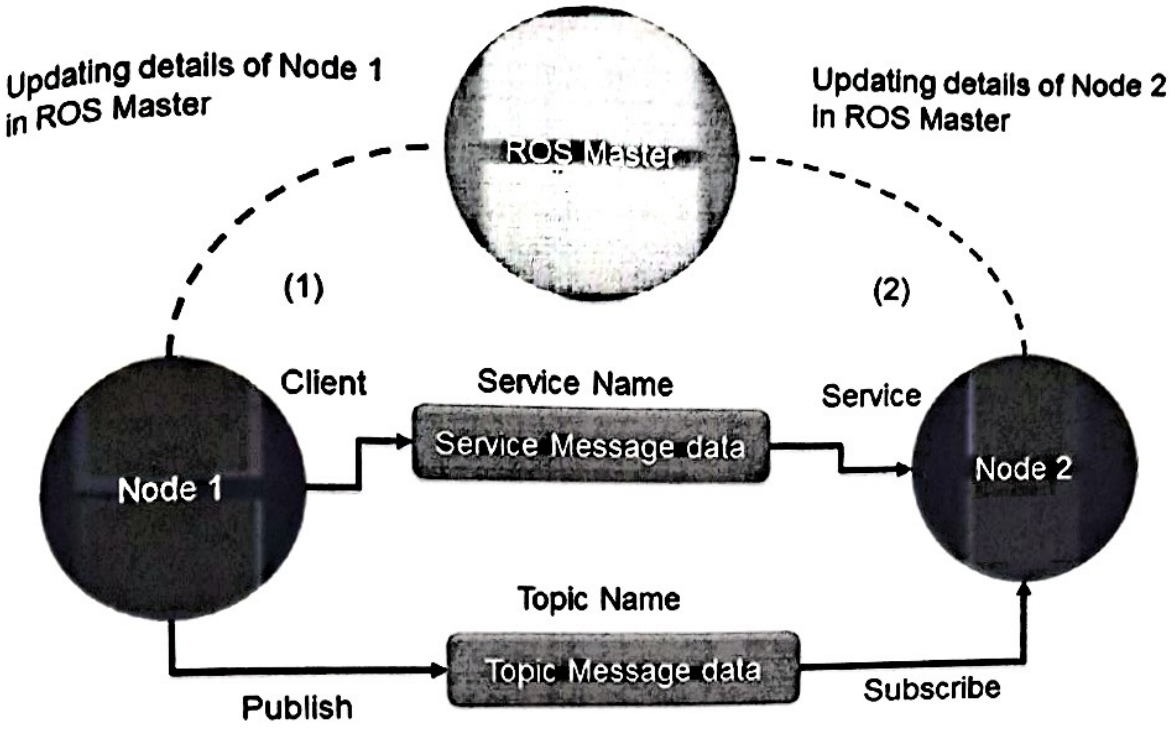


Figure 4-12. *ROS Communication block diagram*

Figure 4-12 shows two programs marked as node 1 and node 2. When any of the programs start, a node communicates to a ROS program called the ROS master. The node sends all its information to the ROS master, including the type of data it sends or receives. The nodes that are sending a data are called *publisher nodes*, and the nodes that are receiving data are called *subscriber nodes*. The ROS Master has all the publisher and subscriber information running on computers. If node 1 sends particular data called "A" and the same data is required by node 2, then the ROS master sends the information to the nodes so that they can communicate with each other.

The ROS nodes can send different types of data to each other, which includes primitive data types such as integer, float, string, and so forth. The different data types being sent are called *ROS messages*. With ROS messages, we can send data with a single data type or multiple data with different data types. These messages are sent through a message bus or path called *ROS topics*. Each topics has a name; for example, a topic named "chatter" sends a string message.

www.electronicbo.com

When a ROS node publishes a topic, it sends a ROS topic with a ROS message, and it has data with the message type.

In Figure 4-12, the ROS topic is publishing and subscribing node 1 and node 2. This process starts when the ROS master exchange the node details to each other.

Next, let's go through some important concepts and terms that are used when working with ROS. They can be classified as three categories: the ROS file system, ROS computation concepts, and the ROS community.

The ROS File System

The ROS file system includes packages, meta packages, package manifests, repositories, message types, and services types.

ROS packages are the individual units, or the *atomic units,* of ROS software. All source code, data files, build files, dependencies, and other files are organized in packages. A ROS meta package groups a set of similar packages for a specific application. A ROS meta package does not have any source files or data files. It has the dependencies of similar packages. A ROS meta package organizes a set of packages.

A *package manifest* is an XML file placed inside a ROS package. It has all the primary information of a ROS package, including the name of the package, description, author, dependencies, and so forth. A typical package.xml is shown next.

```
<?xml version="1.0"?>
<package>
  <name>test_pkg</name>
  <version>0.0.1</version>
  <description>The test package</description>
  <maintainer email="qboticslabs@gmail.com">robot</maintainer>
  <license>BSD</license>
```

```
<buildtool_depend>catkin</buildtool_depend>
.....                    ..... .
<run_depend>catkin</run_depend>
..... .                  ..... .
</package>
```

A *ROS repository* is a collection of ROS packages that share a common version control system.

A *message type description* is the definition of a new ROS message type. There are existing data types available in ROS that can be directly used for our application, but if we want to create a new ROS message, we can. A new message type can be defined and stored inside the `msg` folder inside the package.

Similar to message type, a *service type definition* contains our own service definitions. It is stored in the `srv` folder.

Figure 4-13 shows a typical ROS package folder.

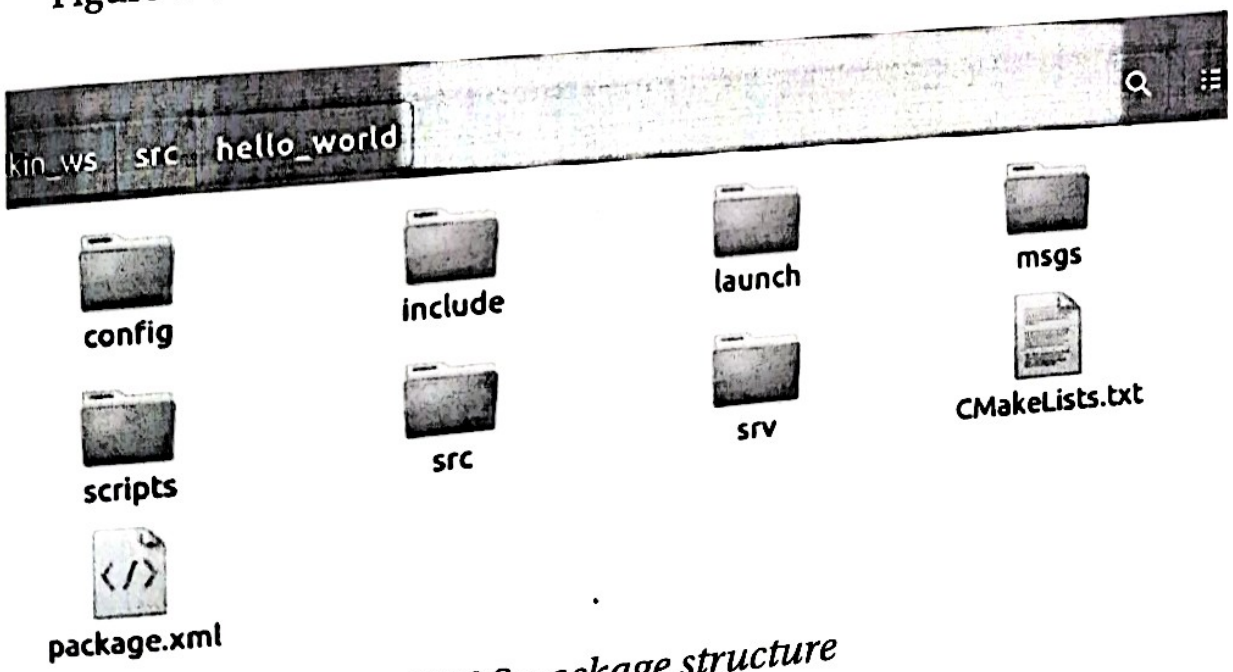


Figure 4-13. *A typical ROS package structure*

www.electronicbo.com

Understanding ROS-Industrial packages

ROS-Industrial basically extends the advanced capabilities of ROS software to industrial robots employed in manufacturing processes. ROS-Industrial consists of many software packages, which help us to control industrial robots. These packages are **BSD (legacy)/Apache 2.0** (preferred) licensed programs, which contain libraries, drivers, and tools with a standard solution for industrial hardware. ROS-Industrial is now guided by the ROS-Industrial Consortium. The official website of ROS-Industrial (ROS-I) can be found at `http://rosindustrial.org/`:

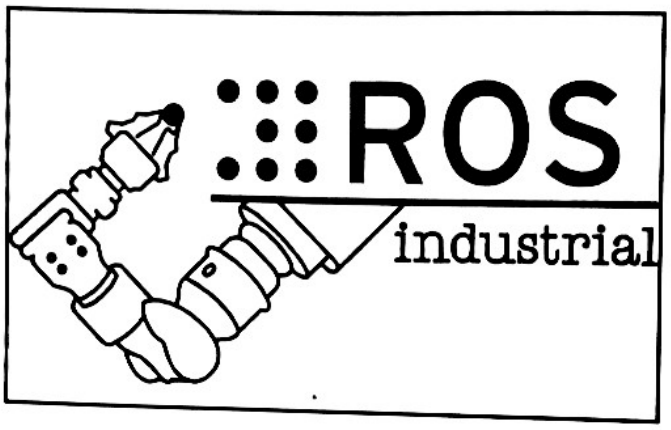


Figure 1: Logo of ROS-Industrial

Goals of ROS-Industrial

The main goals behind ROS-Industrial development are as follows:

- Combining the strength of ROS with the existing industrial technologies to explore the advanced capabilities of ROS in the manufacturing process
- Developing a reliable and robust software for industrial robot applications
- Providing an easy way to do research and development in industrial robotics
- Creating a wide community supported by researchers and professionals for industrial robotics
- Providing industrial-grade ROS applications and becoming a one-stop location of industry-related applications

ROS-Industrial – a brief history

In 2012, the ROS-Industrial open source project started as the collaboration of Yaskawa Motoman Robotics (`http://www.motoman.com/`), Willow Garage (`https://www.willowgarage.com/`), and the **Southwest Research Institute (SwRI)** (`http://www.swri.org/`), for using ROS in industrial manufacturing. The ROS-I was founded by Shaun Edwards in January 2012.

In March 2013, the ROS-I Consortium Americas was launched, led by SwRI, and ROS-I Consortium Europe was launched, led by Fraunhofer IPA in Germany.

Benefits of ROS-Industrial

Let's look at the benefits ROS-I provides to the community:

- **Explores the features in ROS**: The ROS-Industrial packages are tied to the ROS framework so that we can use all the ROS features in industrial robots too. Using ROS, we can create custom IK solvers for each robot and implement object manipulation, using 2D/3D perception.
- **Out-of-the-box applications**: The ROS interface enables advanced perception in robots for working with picking and placing complex objects.
- **Simplifies robotic programming**: ROS-I eliminates teaching and planning paths of robots and, instead, automatically calculates a collision-free optimal path for the given points.
- **Open source**: ROS-I is open source software that allows commercial use without any restrictions.

Installing ROS-Industrial packages

Installing ROS-I packages can be done by using package managers or building from the source code. If we have installed the `ros-kinetic-desktop-full` installation, we can use the following command to install ROS-Industrial packages on Ubuntu, working with ROS Kinetic:

```
$ sudo apt-get install ros-kinetic-industrial-core
```

↓

adv package tool (CL)

The preceding command will install the core packages of ROS-Industrial packages. The `industrial-core` stack includes the following set of ROS packages:

- `industrial-core`: This stack contains packages and libraries for supporting industrial robotic systems. The package consists of nodes for communicating with industrial robot controllers and industrial robot simulators, and also provides ROS controllers for industrial robots.
- `industrial_deprecated`: This package contains nodes, launch files, and so on that are going to be deprecated. The files inside this package could be deleted from the repository in the next ROS versions, so we should look for the replacement of these files before the content is going to be deleted.
- `industrial_msgs`: This package contains message definitions, which are specific to the ROS-Industrial packages.
- `simple_message`: This is a part of ROS-Industrial packages that is a standard message protocol containing a simple messaging framework for communicating with industrial robot controllers.
- `industrial_robot_client`: This package contains a generic robot client for connecting to industrial robot controllers, which is running an industrial robot server and can communicate using a simple message protocol.
- `industrial_robot_simulator`: This package simulates the industrial robot controller, which follows the ROS-Industrial driver standard. Using this simulator, we can simulate and visualize the industrial robot.
- `industrial_trajectory_filters`: This package contains libraries and plugins for filtering the trajectories, which are sent to the robot controller.

Block diagram of ROS-Industrial packages

The following diagram is a simple block diagram representation of ROS-I packages, which are organized on top of ROS. We can see the ROS-I layer on top of the ROS layers. We can see a brief description of each of the layers for better understanding. The following diagram is taken from the ROS-I wiki page (`http://wiki.ros.org/Industrial`):

- **The ROS GUI**: This layer includes the ROS plugin-based GUI tools layer, which consists of tools such as RViz, `rqt_gui`, and so on
- **The ROS-I GUI**: These GUIs are standard industrial UIs for working with industrial robots that may be implemented in the future
- **The ROS Layer**: This is the base layer in which all communications are taking place

- **The MoveIt! Layer**: The MoveIt! layer provides a direct solution to industrial manipulators in planning, kinematics, and pick and place
- **The ROS-I Application Layer**: This layer consists of an industrial process planner, which is used to plan what is to be manufactured, how it will be manufactured, and what resources are needed for the manufacturing process
- **The ROS-I Interface Layer**: This layer consists of the industrial robot client, which can be connected to the industrial robot controller using the simple message protocol
- **The ROS-I Simple Message Layer**: This is the communication layer of the industrial robot, which is a standard set of protocols that will send data from the robot client to the controller and vice versa
- **The ROS-I Controller Layer**: This layer consists of vendor-specific industrial robot controllers

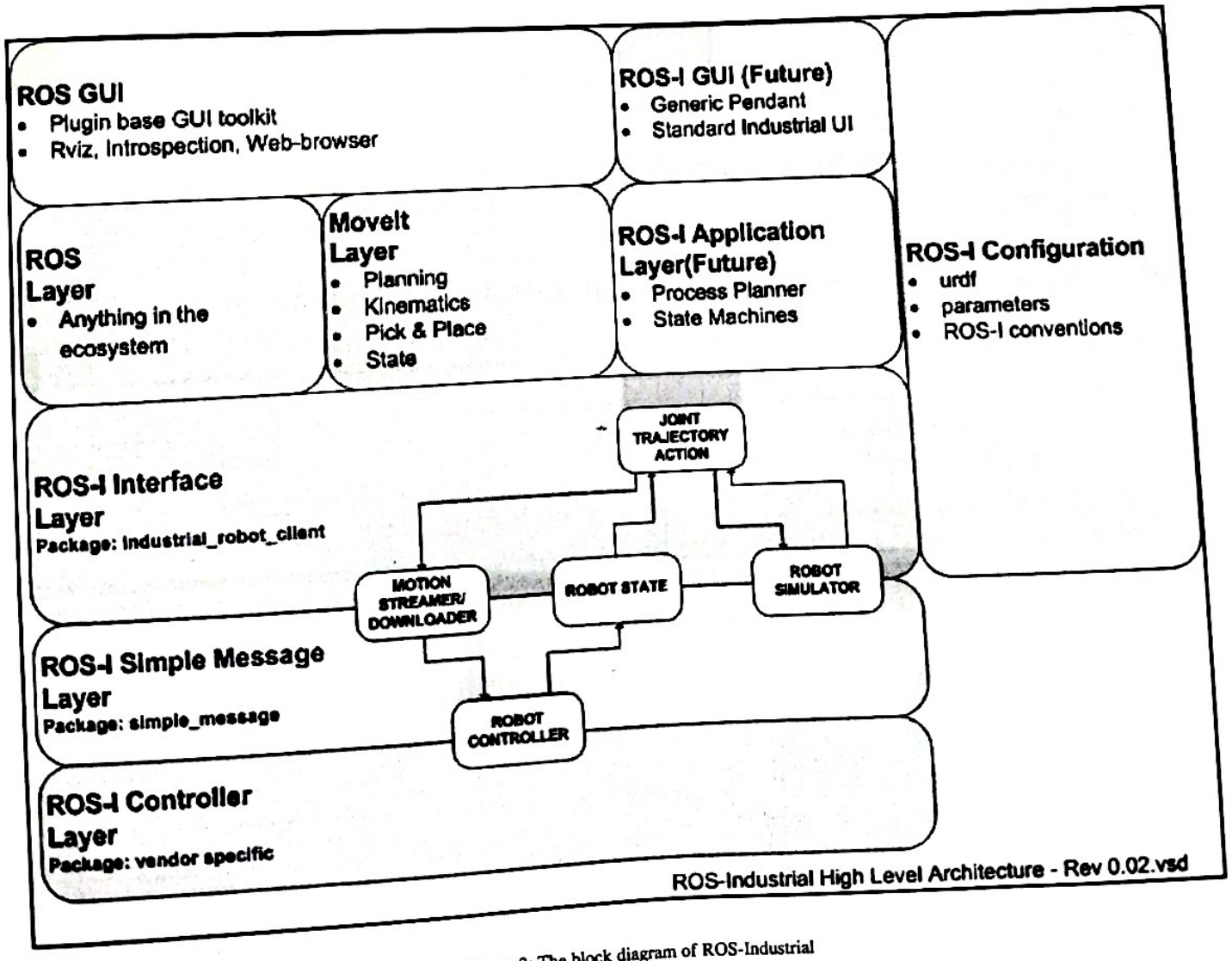


Figure 2: The block diagram of ROS-Industrial

After discussing the basic concepts, we will start to interface an industrial robot to ROS using ROS-Industrial. Firstly, we will show how to create a URDF model of an industrial robot and how to create a proper MoveIt! configuration for it. Then, we will discuss how to control real and simulated Universal Robots and Abb industrial manipulators, analyzing all the necessary elements of a ROS-I package. Finally, we will work with the Ikfast algorithm and plugin to speed up kinematic calculation with MoveIt!.

Creating a URDF for an industrial robot

Creating the URDF file for an ordinary robot and an industrial robot are the same, but industrial robots require some standards that should be strictly followed during their URDF modeling, which are as follows:

- **Simplify the URDF design**: The URDF file should be simple and readable and only need the important tags
- **Develop a common design**: Develop a common design formula for all industrial robots by various vendors
- **Modularize the URDF**: The URDF needs to be modularized using XACRO macros and it can be included in a large URDF file without much hassle

The following points are the main differences in the URDF design followed by ROS-I.

- **Collision-aware**: The industrial robot IK planners are collision-aware, so the URDF should contain an accurate collision 3D mesh for each link. Every link in the robot should export to STL or DAE with a proper coordinate system. The coordinate system that ROS-I is following is X-axes pointing forward and Z-axes pointing up when each joint is in the zero position. It is also to be noted that, if the joint's origin coincides with the base of the robot, the transformation will be simpler. It will be good if we are putting robot-based joints in the zero position (origin), which can simplify the robot design. In ROS-I, the mesh file used for visual purposes is highly detailed, but the mesh file used for collision will not be detailed, because it takes more time to perform collision checking. In order to remove the mesh details, we can use tools such as MeshLab (`http://meshlab.sourceforge.net/`), using its option (`Filters -> Remeshing, Simplification and Reconstruction -> Convex Hull`).

- **URDF joint conventions**: The orientation value of each robot joint is limited to single rotation, that is, out of the two orientation (`roll`, `pitch`, and `yaw`) values, only one value will be there.
- **Xacro macros**: In ROS-I, the entire manipulator section is written as a macro using `xacro`. We can add an instance of this macro in another macro file, which can be used for generating a URDF file. We can also include additional end-effector definitions on this same file.
- **Standards frames**: In ROS-I, the `base_link` frame should be the first link and `tool0` (tool-zero) should be the end-effector link. Also, the `base` frame should match with the `base` of the robot controller. In most cases, the transform from `base` to `base_link` is treated as fixed.

After building the `xacro` file for the industrial robot, we can convert to URDF and verify it using the following command:

```
$ rosrun xacro xacro -inorder -o <output_urdf_file> <input_xacro_file>
$ check_urdf <urdf_file>
```

Next, we can discuss the differences in creating the MoveIt! configuration for an industrial robot.